10.4 @ElementCollection

Although it is not an entity association type, the `@ElementCollection` is very similar to the unidirectional `@OneToMany` relationship. To represent collections of basic types (e.g. `String`, `int`, `BigDecimal`) or embeddable types, the `@ElementCollection` must be used instead. If the previous associations involved multiple entities, this time, there is only a single `Post` entity with a collection of `String` comments.

Value types inherit the persistent state from their parent entities, so their lifecycle is also bound to the owner entity. Any operation against the entity collection is going to be automatically materialized into a DML statement.

From a database perspective, there is one child table holding both the foreign key column and the collection element value.

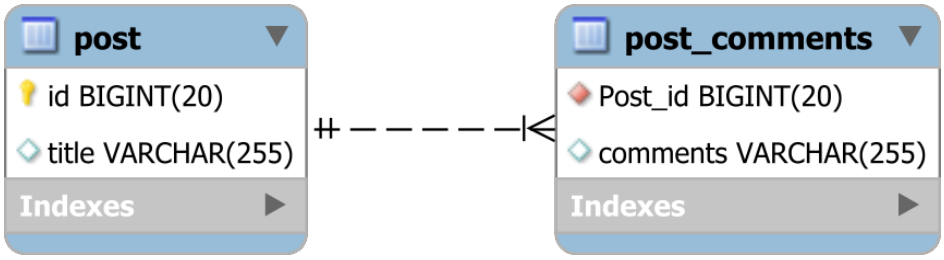

Figure 10.7: The @ElementCollection table relationship

10.4.1 @ElementCollection List

Let's consider the use case when a `List` is used with `@ElementCollection`:

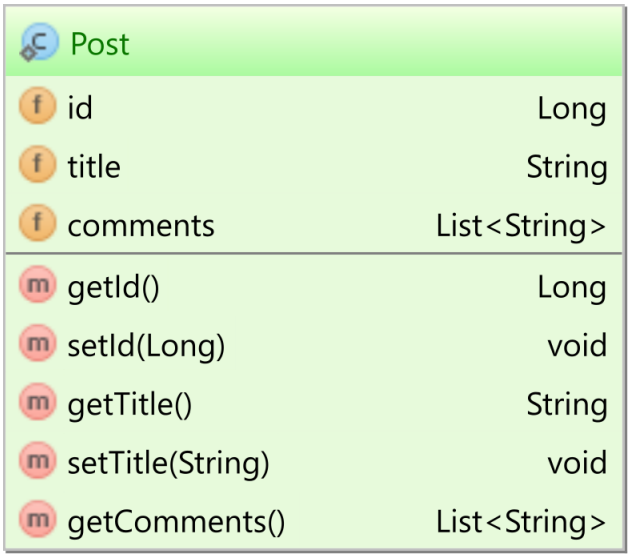

Figure 10.8: The @ElementCollection relationship

The mapping for the `comments` collection looks as follows:

```
@ElementCollection
private List<String> comments = new ArrayList<>();
```

When it comes to adding or removing child records, the `@ElementCollection` behaves like a unidirectional `@OneToMany` relationship, annotated with `CascadeType.ALL` and `orphanRemoval`.

To persist three comments, the data access layer only has to add them to the parent entity collection:

```
post.getComments().add("My first review");
post.getComments().add("My second review");
post.getComments().add("My third review");
```

Hibernate issues the insert statements during Persistence Context flushing:

```
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My first review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```

Unfortunately, the remove operation uses the same logic as the unidirectional `@OneToMany` association, so when removing the first collection element:

```
post.getComments().remove(0);
```

Hibernate deletes all the associated child-side records and re-inserts the in-memory ones back into the database table:

```
DELETE FROM Post_comments WHERE Post_id = 1
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My second review')
INSERT INTO Post_comments (Post_id, comments) VALUES (1, 'My third review')
```

Using an `@ElementCollection` with a `List` is not very efficient because the association is considered to be a bag, in Hibernate terminology.

A bag does not guarantee that elements are uniquely identifiable, hence Hibernate needs to delete and reinsert the elements associated with a given parent entity whenever a change occurs to the `@ElementCollection` `List`.

10.4.2 @ElementCollection Set

On the other hand, when using a `Set`, the `@ElementCollection` no longer behaves like a bag, and the generated SQL statements are going to be more efficient.

So, let's changing the `comments` collection to use a `Set` this time:

```
@ElementCollection
private Set<String> comments = new HashSet<>();
```

After inserting the same 3 comments, as we did when using a `List` with `@ElementCollection`, let's see how the `Set` behaves when it comes to removing elements.

When removing the first element, we added to the `Set`:

```
post.getComments().remove("My first review");
```

Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My first review'
```

When removing the last element, we added to the `Set`:

```
post.getComments().remove("My third review");
```

Again, Hibernate generates a single SQL delete statement:

```
DELETE FROM Post_comments WHERE Post_id = 1 and comments = 'My third review'
```

So, if you want to use an `@ElementCollection` and benefit from its simplicity, it's much better if you are using a `Set`, instead of a `List`.

However, even if using a `Set` can render more efficient SQL statements than when using a `List`, it's still much better if you replace the `@ElementCollection` with either a bidirectional `@OneToMany` association or just a simple `@ManyToOne` association that maps the Foreign Key column.

This way, you can remove elements without having to fetch the collection via the parent entity.

10.5 @OneToOne

From a database perspective, the one-to-one association is based on a foreign key that is constrained to be unique. This way, a parent row can be referenced by at most one child record only.

In JPA, the `@OneToOne` relationship can be either unidirectional or bidirectional.

10.5.1 Unidirectional @OneToOne

In the following example, the `Post` entity represents the parent-side, while the `PostDetails` is the child-side of the one-to-one association.

As already mentioned, the JPA entity relationship diagram matches exactly the one-to-one table relationship.

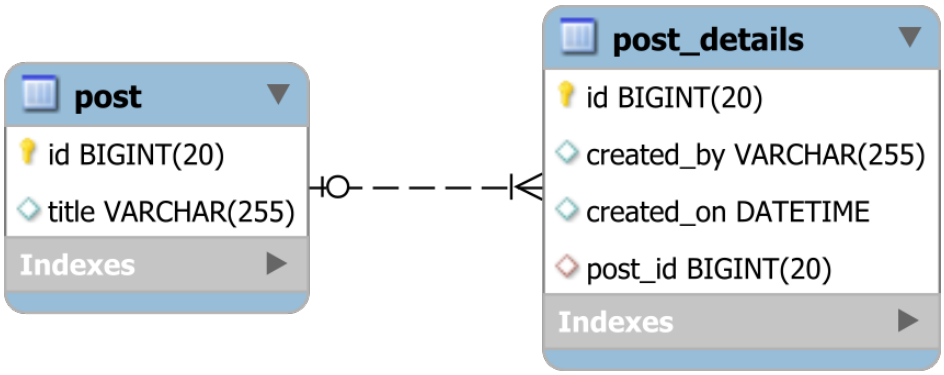

Figure 10.9: The one-to-one table relationship

Even from the Domain Model side, the unidirectional `@OneToOne` relationship is strikingly similar to the unidirectional `@ManyToOne` association.

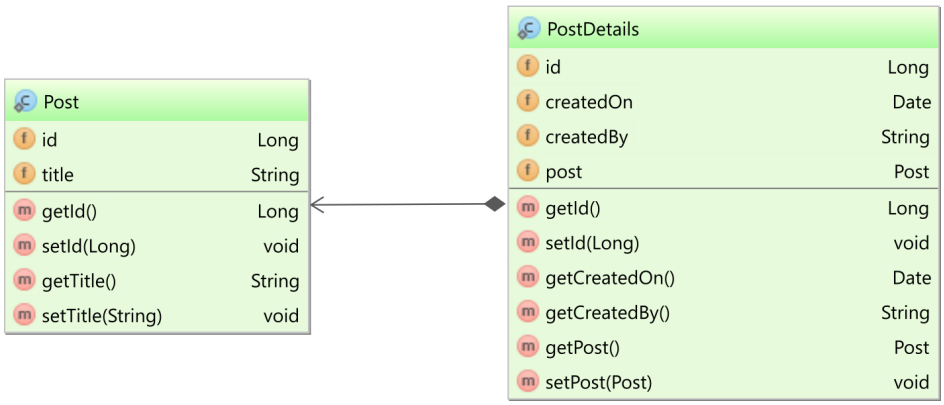

Figure 10.10: The unidirectional @OneToOne relationship

The mapping is done through the `@OneToOne` annotation, which, just like the `@ManyToOne` mapping, might also take a `@JoinColumn` as well.

```
@OneToOne
@JoinColumn(name = "post_id")
private Post post;
```

The unidirectional `@OneToOne` association controls the associated foreign key, so, when the `post` attribute is set:

```
Post post = entityManager.find(Post.class, 1L);
PostDetails details = new PostDetails("John Doe");
details.setPost(post);
entityManager.persist(details);
```

Hibernate populate the foreign key column with the associated `post` identifier:

```
INSERT INTO post_details (created_by, created_on, post_id, id)
VALUES ('John Doe', '2016-01-08 11:28:21.317', 1, 2)
```

Even if this is a unidirectional association, the `Post` entity is still the parent-side of this relationship. To fetch the associated `PostDetails`, a JPQL query is needed:

```
PostDetails details = entityManager.createQuery(
    "select pd " +
    "from PostDetails pd " +
    "where pd.post = :post", PostDetails.class)
.setParameter("post", post)
.getSingleResult();
```

If the `Post` entity always needs its `PostDetails`, a separate query might not be desirable. To overcome this limitation, it is important to know the `PostDetails` identifier prior to loading the entity.

One workaround would be to use a `@NaturalId`, which might not require a database access if the entity is stored in the second-level cache. Fortunately, there is even a simpler approach which is also portable across JPA providers as well. The JPA 2.0 specification added support for derived identifiers, making possible to link the `PostDetails` identifier to the `post` table primary key.

This way, the `post_details` table primary key can also be a foreign key referencing the `post` table identifier.

The `PostDetails` `@OneToOne` mapping is changed as follows:

```
@OneToOne
@MapsId
private Post post;
```

This time, the table relationship does not feature any additional foreign key column since the `post_details` table primary key references the `post` table primary key:

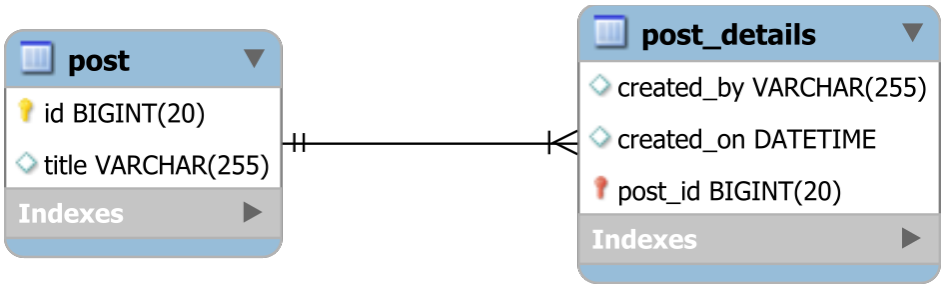

Figure 10.11: The shared key one-to-one

Because `PostDetails` has the same identifier as the parent `Post` entity, it can be fetched without having to write a JPQL query.

```
PostDetails details = entityManager.find(PostDetails.class, post.getId());
```

The shared primary key efficiency

First of all, the shared primary key approach reduces the memory footprint of the child-side table indexes since it requires a single indexed column instead of two. The more records a child table has, the better the improvement gain for reducing the number of indexed columns.

More, the child entity can now be simply retrieved from the second-level cache, therefore preventing a database hit. In the previous example, because the child entity identifier was not known, a query was inevitable. To optimize the previous use case, the *query cache* would be required as well, but the query cache is not without issues either.

Because of the reduced memory footprint and enabling the second-level cache direct retrieval, the JPA 2.0 *derived identifier* is the preferred `@OneToOne` mapping strategy. The shared primary key is not limited to unidirectional associations, being available for bidirectional `@OneToOne` relationships as well.

10.5.2 Bidirectional @OneToOne

A bidirectional `@OneToOne` association allows the parent entity to map the child-side as well:

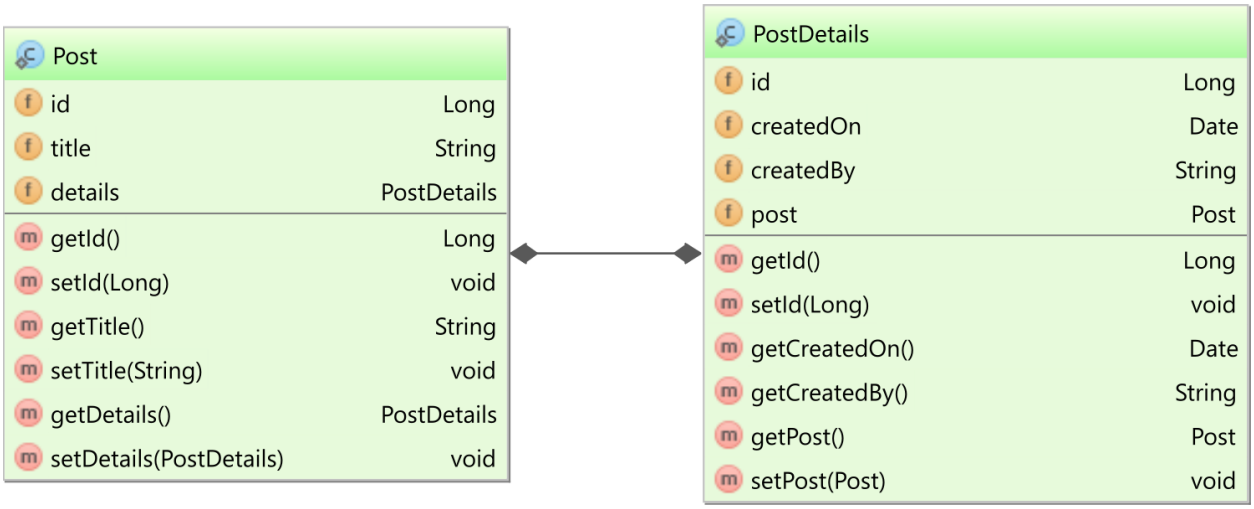

Figure 10.12: The bidirectional @OneToOne relationship

The parent-side defines a `mappedBy` attribute because the child-side (which can still share the primary key with its parent) is still in charge of this JPA relationship:

```
@OneToOne(mappedBy = "post", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private PostDetails details;
```

Because this is a bidirectional relationship, the `Post` entity must ensure that both sides of this relationship are set upon associating a `PostDetails` entity:

```
public void setDetails(PostDetails details) {
    if (details == null) {
        if (this.details != null) this.details.setPost(null);
    }
    else details.setPost(this);
    this.details = details;
}
```

Unlike the parent-side `@OneToMany` relationship where Hibernate can simply assign a proxy even if the child collection is empty, the `@OneToOne` relationship must decide if to assign the child reference to null or to an `Object`, be it the actual entity object type or a runtime Proxy.

This is an issue that affects the parent-side `@OneToOne` association, while the child-side, which has an associated foreign key column, knows whether the parent reference should be null or not. For this reason, the parent-side must execute a secondary query to know if there is a mirroring foreign key reference on the child-side.

Even if the association is lazy, when fetching a `Post` entity:

```
Post post = entityManager.find(Post.class, 1L);
```

Hibernate fetches the child entity as well, so, instead of only one query, Hibernate requires two select statements:

```
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_
FROM   post p
WHERE  p.id = 1

SELECT pd.post_id AS post_id3_1_0_, pd.created_by AS created_1_1_0_,
       pd.created_on AS created_2_1_0_
FROM   post_details pd
WHERE  pd.post_id = 1
```

If the application developer only needs parent entities, the additional child-side secondary queries will be executed unnecessarily, and this might affect application performance. The more parent entities are needed to be retrieved, the more obvious the secondary queries performance impact gets.

Limitations

Even if the foreign key is `NOT NULL` and the parent-side is aware about its non-nullability through the `optional` attribute (e.g. `@OneToOne(mappedBy = "post", fetch = FetchType.LAZY, optional = false)`), Hibernate still generates a secondary select statement.

For every managed entity, the Persistence Context requires both the entity type and the identifier, so the child identifier must be known when loading the parent entity, and the only way to find the associated `post_details` primary key is to execute a secondary query. Because the child identifier is known when using `@MapsId`, in future, HHH-10771[a] should address the secondary query issue.

Bytecode enhancement is the only viable workaround. However, it only works if the parent side is annotated with `@LazyToOne(LazyToOneOption.NO_PROXY)` and the child side is not using `@MapsId`. Because it's simpler and more predictable, the unidirectional `@OneToOne` relationship is often preferred.

[a] https://hibernate.atlassian.net/browse/HHH-10771

10.6 @ManyToMany

From a database perspective, the `@ManyToMany` annotation mirrors a many-to-many table relationship:

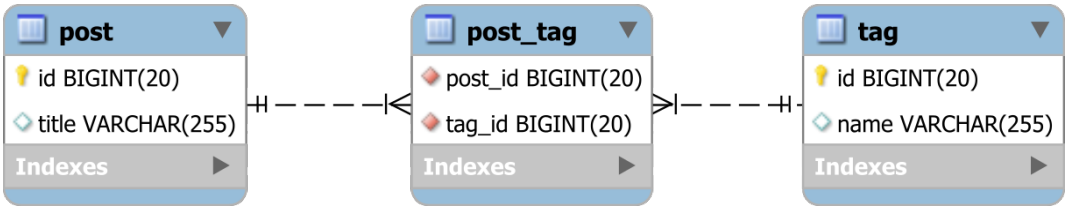

Figure 10.13: The many-to-many table relationship

Like the `@OneToMany` and `@OneToOne` relationships, the `@ManyToMany` association can be either unidirectional or bidirectional.

10.6.1 Unidirectional @ManyToMany List

In our case, if we need to map a unidirectional `@ManyToMany` association between the `Post` and `Tag` entities, it makes sense to have the `Post` entity map the `@ManyToMany` relationship since there is not much need for navigating this association from the `Tag` side (although we can still do it with a JPQL query).

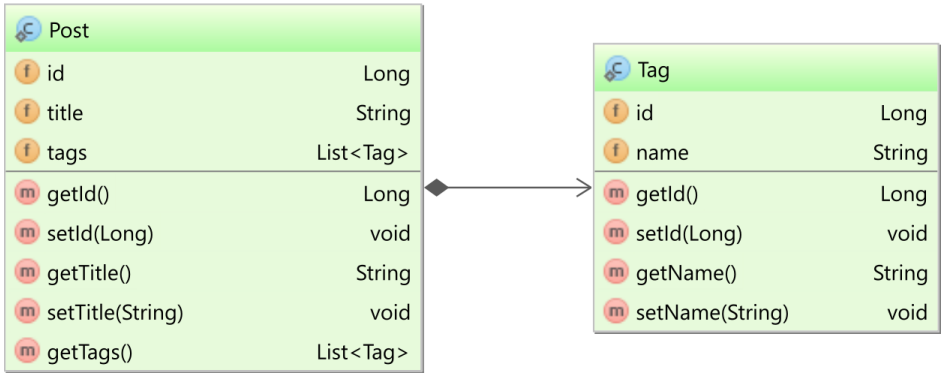

Figure 10.14: The unidirectional @ManyToMany relationship

In the `Post` entity, the `tags` collection using a `List`, like this:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private List<Tag> tags = new ArrayList<>();
```

When adding several entities:

```
Post post1 = new Post("JPA with Hibernate");
Post post2 = new Post("Native Hibernate");

Tag tag1 = new Tag("Java");
Tag tag2 = new Tag("Hibernate");

post1.getTags().add(tag1);
post1.getTags().add(tag2);
post2.getTags().add(tag1);

entityManager.persist(post1);
entityManager.persist(post2);
```

Hibernate manages to persist both the `Post` and the `Tag` entities along with their junction records.

```
INSERT INTO post (title, id) VALUES ('JPA with Hibernate', 1)
INSERT INTO post (title, id) VALUES ('Native Hibernate', 4)

INSERT INTO tag (name, id) VALUES ('Java', 2)
INSERT INTO tag (name, id) VALUES ('Hibernate', 3)

INSERT INTO post_tag (post_id, tag_id) VALUES (1, 2)
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
INSERT INTO post_tag (post_id, tag_id) VALUES (4, 2)
```

Cascading

For `@ManyToMany` associations, `CascadeType.REMOVE` does not make too much sense when both sides represent independent entities. In this case, removing a `Post` entity should not trigger a `Tag` removal because the `Tag` can be referenced by other *posts* as well. The same arguments apply to orphan removal since removing an entry from the `tags` collection should only delete the junction record and not the target `Tag` entity.

For both unidirectional and bidirectional associations, it is better to avoid the `CascadeType.REMOVE` mapping. Instead of `CascadeType.ALL`, the cascade attributes should be declared explicitly (e.g. `CascadeType.PERSIST`, `CascadeType.MERGE`).

But just like the unidirectional `@OneToMany` association, problems arise when it comes to removing the junction records from the `List`:

```
post1.getTags().remove(tag1);
```

Hibernate deletes all junction rows associated with the `Post` entity whose `Tag` association is being removed and inserts back the remaining ones:

```
DELETE FROM post_tag WHERE post_id = 1
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
```

10.6.2 Unidirectional @ManyToMany Set

When using a `List`, the `@ManyToMany` association behaves like a bag, and, as explained in the `@ElementCollection` section, Hibernate bags are not very efficient.

Therefore, let's change the `tags` collection to use a `Set`:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

Assuming, we added the same 3 `Tag` entities to the `Post` entity, if we remove the first `Tag` from the `tags` collection:

```
post1.getTags().remove(tag1);
```

Hibernate is going to generate a single SQL delete statement, this time:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 2
```

For a `@ManyToMany` association, it's better to use the `Set` collection type, instead of `List`.

10.6.3 Bidirectional @ManyToMany

The bidirectional `@ManyToMany` relationship can be navigated from both the `Post` and the `Tag` side.

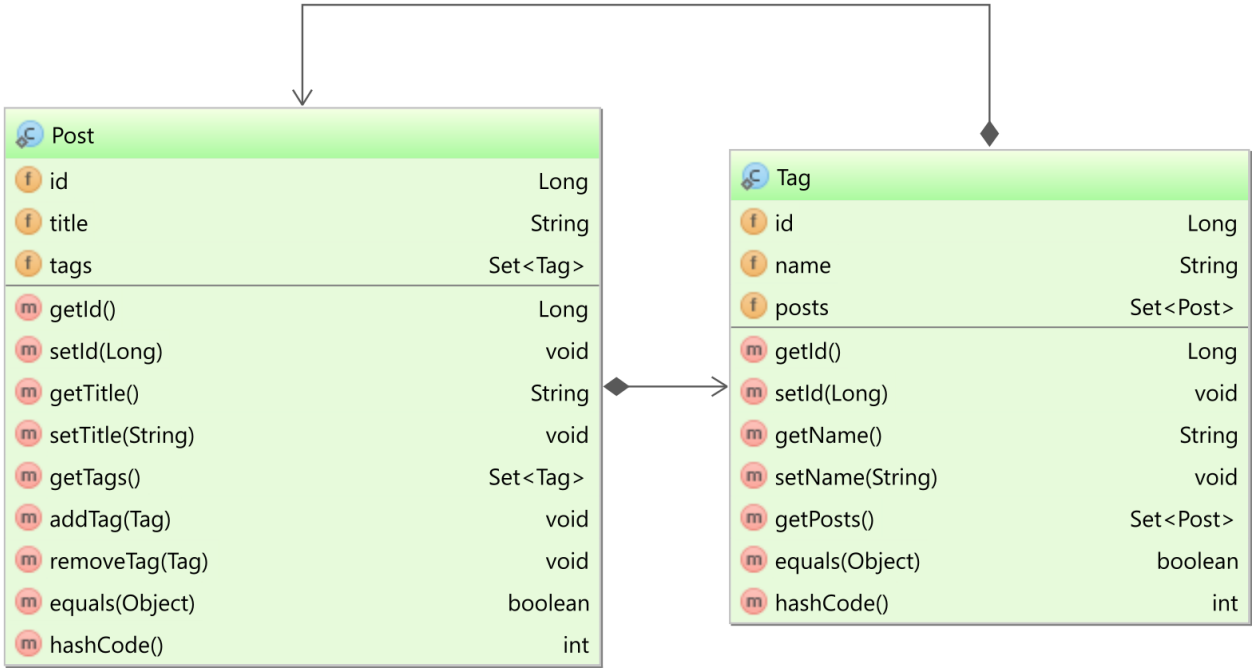

Figure 10.15: The bidirectional @ManyToMany relationship

While in the one-to-many and many-to-one associations the child-side is the one holding the foreign key, for a many-to-many table relationship both ends are parent-sides and the junction table plays the child-side role.

Because the junction table is hidden when using the default `@ManyToMany` mapping, the application developer must choose an owning and a `mappedBy` side.

The `Post` entity uses the same mapping as shown in the unidirectional `@ManyToMany` section:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<Tag> tags = new HashSet<>();
```

while the `Tag` entity adds a `@ManyToMany` association that looks as follows:

```
@ManyToMany(mappedBy = "tags")
private Set<Post> posts = new HashSet<>();
```

The `mappedBy` attribute tells Hibernate that the `Post` entity is the owning side of this association. In our case, the `mappedBy` attribute of the `@ManyToMany` annotation in the `Tag` entity points to the `tags` collection of the `Post` entity, meaning that the `Post` entity is going to propagate the association state changes to the underlying junction table.

Hibernate manages each side of a bidirectional `@ManyToMany` relationship just like a unidirectional `@OneToMany` association between the parent-side (e.g. `Post` or the `Tag`) and the hidden child-side (e.g. the `post_tag` table `post_id` or `tag_id` foreign keys).

For this reason, it's much more efficient to use a `Set` collection type, no matter if the `@ManyToMany` is unidirectional or bidirectional.

Like any bidirectional associations, helper methods are needed to keep both parent sides in sync. For a bidirectional `@ManyToMany` association, the helper methods must be added to the entity that is more likely to interact with. In our case, the root entity is the `Post`, so the helper methods are added to the `Post` entity:

```
public void addTag(Tag tag) {
    tags.add(tag);
    tag.getPosts().add(this);
}

public void removeTag(Tag tag) {
    tags.remove(tag);
    tag.getPosts().remove(this);
}
```

Both `Post` and `Tag` entities have unique attributes which can simplify the entity removal operation even when mixing detached and managed entities.

10.6.4 The @OneToMany alternative

We can also map a many-to-many table relationship using bidirectional `@OneToMany` associations from the parent entities to the child entity that maps the junction table.

Therefore, the `post_tag` child table is mapped by the `PostTag` entity, and its composite identifier is encapsulated in the `PostTagId` embeddable type as illustrated by the following diagram.

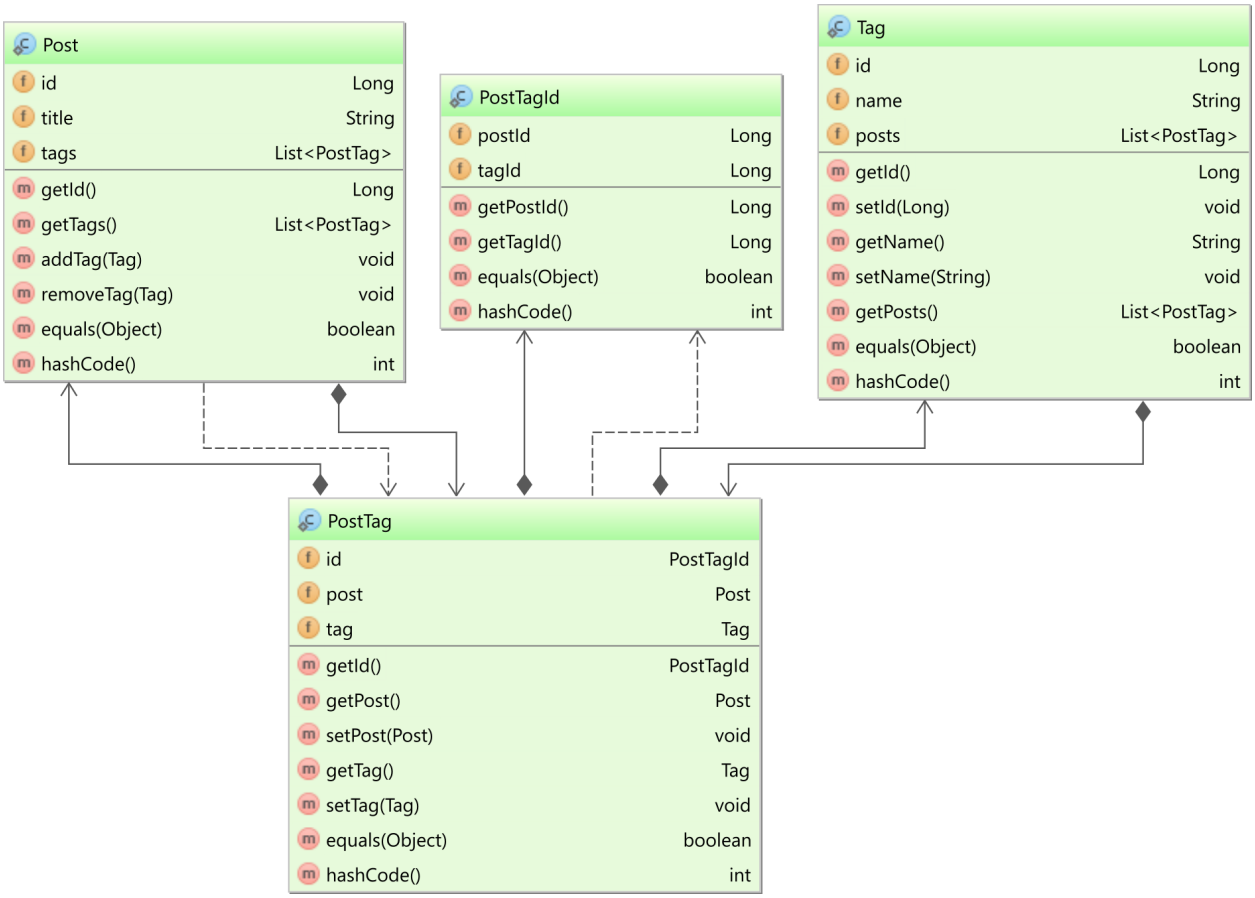

Figure 10.16: The @OneToMany as a many-to-many table relationship

The `PostTagId` maps the `postId` and `tagId` foreign key columns, and the `PostTag` entity uses the `postId` and `tagId` properties of the `PostTagId` identifier to map the `post` and the `tag` `@ManyToOne` associations.

The `Post` entity maps the bidirectional `@OneToMany` side of the `post` `@ManyToOne` association from the `PostTag` child entity:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> tags = new ArrayList<>();
```

The `Tag` entity maps the bidirectional `@OneToMany` side of the `tag` `@ManyToOne` association defined by the `PostTag` child entity:

```
@OneToMany(mappedBy = "tag", cascade = CascadeType.ALL, orphanRemoval = true)
private List<PostTag> posts = new ArrayList<>();
```

This way, the bidirectional `@ManyToMany` relationship is transformed in two bidirectional `@OneToMany` associations.

The `PostTag` entity has a composed identifier made out of the `post_id` and `tag_id` columns.

```
@Embeddable
public class PostTagId implements Serializable {

    @Column(name = "post_id")
    private Long postId;

    @Column(name = "tag_id")
    private Long tagId;

    public PostTagId() {}

    public PostTagId(Long postId, Long tagId) {
        this.postId = postId;
        this.tagId = tagId;
    }

    public Long getPostId() {
        return postId;
    }

    public Long getTagId() {
        return tagId;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTagId that = (PostTagId) o;
        return Objects.equals(postId, that.getPostId() &&
                Objects.equals(tagId, that.getTagId());
    }

    @Override
    public int hashCode() {
        return Objects.hash(postId, tagId);
    }
}
```

The JPA specification requires the composite identifiers embeddable types to implement the `Serializable` interface, as well as implementing `equals` and `hashCode` based on the associated foreign key column values.

The `PostTag` entity defines two `@ManyToOne` associations, as follows:

```
@Entity
@Table(name = "post_tag")
public class PostTag {

    @EmbeddedId
    private PostTagId id;

    @ManyToOne
    @MapsId("postId")
    private Post post;

    @ManyToOne
    @MapsId("tagId")
    private Tag tag;

    private PostTag() {}

    public PostTag(Post post, Tag tag) {
        this.post = post;
        this.tag = tag;
        this.id = new PostTagId(post.getId(), tag.getId());
    }

    //Getters and setters omitted for brevity

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PostTag that = (PostTag) o;
        return Objects.equals(post, that.getPost() &&
                Objects.equals(tag, that.getTag());
    }

    @Override
    public int hashCode() {
        return Objects.hash(post, tag);
    }
}
```

The `@MapsId` annotation instructs Hibernate that the `post_id` and `tag_id` foreign key columns are managed by the `PostTagId` composite identifier, not by the `@ManyToOne` property.

Just like with any bidirectional association, we need to define the helper methods that allow us to keep the `@ManyToOne` and `@OneToMany` sides in sync.

However, as opposed to a standard bidirectional `@OneToMany` association, the `removeTag` helper method in the `Post` entity is much more complex because it needs to locate the `PostTag` associated with the current `Post` entity, as well as the `Tag` object reference in the `tags` collection that needs to be disassociated.

```
public void removeTag(Tag tag) {
    for (Iterator<PostTag> iterator = tags.iterator(); iterator.hasNext(); ) {
        PostTag postTag = iterator.next();
        if (postTag.getPost().equals(this) && postTag.getTag().equals(tag)) {
            iterator.remove();
            postTag.getTag().getPosts().remove(postTag);
            postTag.setPost(null);
            postTag.setTag(null);
            break;
        }
    }
}
```

Notice that, the `removeTag` method relies on the `PostTag` `equals` and `hashCode` methods in order to remove the desired entry.

When rerunning the entity removal example featured in the unidirectional `@ManyToMany` section:

```
post1.removeTag(tag1);
```

Hibernate issues a single delete statement, therefore targeting a single `PostTag` junction record:

```
DELETE FROM post_tag WHERE post_id = 1 AND tag_id = 3
```

Because the `@ManyToOne` side manages the foreign key column changes, the internal collection state of the `@OneToMany` side is not taken into consideration unless we remove an entry.

For this reason, reversing the order of the `tags` elements has absolutely no effect:

```
post1.getTags().sort(
    Comparator.comparing(
        (PostTag postTag) -> postTag.getId().getTagId()
    )
    .reversed()
);
```

To materialize the order of elements, the `@OrderColumn` must be used instead:

```
@OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphanRemoval = true)
@OrderColumn(name = "entry")
private List<PostTag> tags = new ArrayList<>();
```

When using the aforementioned mapping, rhe `post_tag` junction table features an `entry` column storing the collection element order.

Therefore, if we reverse the element order as we did previously, Hibernate is going to update the `entry` column to maintain the current `PostTag` element order:

```
UPDATE post_tag SET entry = 0 WHERE post_id = 1 AND tag_id = 4
UPDATE post_tag SET entry = 1 WHERE post_id = 1 AND tag_id = 3
```

Mapping the junction entity of a many-to-many table relationship allows us to persist additional column into the junction table (e.g. `created_on`, `created_by`), which cannot be done when using the JPA `@ManyToMany` annotation.

10.7 Hypersistence Optimizer

Knowing which are the most efficient associations is of paramount importance when it comes to designing a high-performance data access layer. But choosing the right mappings is not sufficient. You also have to check the Hibernate configuration properties, as well as the queries that get executed.

For this purpose, I created the Hypersistence Optimizer[1] tool. While you could manually investigate the JPA and Hibernate mappings and configurations, having a tool to do this task on your behalf is going to save you a lot of time.

More, Hypersistence Optimizer can validate your JPA and Hibernate usage during testing, and you could even assert the number of issues detected by the tool and trigger a build failure whenever a performance-impacting issue is detected. This way, you can ensure that future data access logic changes won't affect the performance of your application.

Because you purchased this book, you have a **25$** discount for any Hypersistence Optimizer license. All you have to do is to use the **HO25OFF** coupon code when making the purchase. Or, you can use this link[2] to quickly navigate to the Hypersistence Optimizer sales page with the discount coupon activated.

10.7.1 Testimonials

> "It really pays off when it comes to analyzing complex applications. For architectures that are difficult to manage, it quickly provides clues for further analysis.
>
> This is a huge help both when setting up a new implementation and when optimizing a legacy application."
>
> – Kevin Peters (Software Engineer - codecentric AG)

> "Let's face it, JPA and Hibernate are very powerful but not simple tools. Even experienced developers make mistakes that may result in very heavy database queries, dramatically lowering overall application performance.
>
> Hypersistence Optimizer feels like an additional team member - a JPA expert that is there to help and show how can you optimize your mappings and configurations before you ship them to production."
>
> – Maciej Walkowiak (Freelance Tech Lead)

[1] https://vladmihalcea.com/hypersistence-optimizer/
[2] https://vladmihalcea.teachable.com/p/hypersistence-optimizer/?coupon_code=HO25OFF

11. Inheritance

Java, like any other object-oriented programming language, makes heavy use of inheritance and polymorphism. Inheritance allows defining class hierarchies that offer different implementations of a common interface.

Conceptually, the Domain Model defines both data (e.g. persisted entities) and behavior (business logic). Nevertheless, inheritance is more useful for varying behavior rather than reusing data (composition is much more suitable for sharing structures). Even if the data (persisted entities) and the business logic (transactional services) are decoupled, inheritance can still help varying business logic (e.g. Visitor pattern[1]).

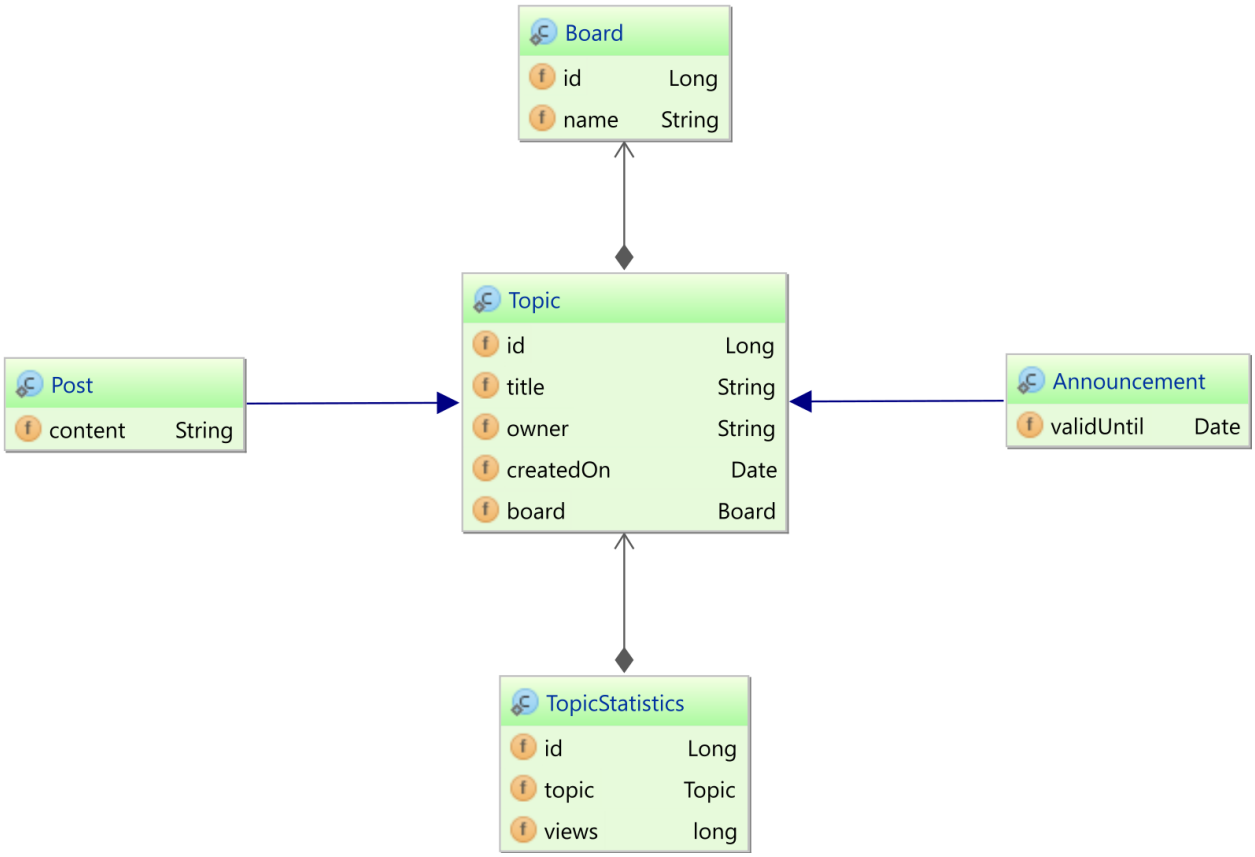

Figure 11.1: Domain Model Inheritance

[1] https://en.wikipedia.org/wiki/Visitor_pattern